CP312 ASSIGNMENT 4

I certify that this Assignment is entirely my work and that all external sources, if used, have been properly acknowledged.

1.

```
MinTowers(P, n, R):
    Sort P in non-decreasing order
    Towers = {}
    i = 1
    while i <= n:
        coverage_limit = P[i] + R
        while i <= n and p[i] <= coverage_limit:
            last_house_covered_by_left_side = P[i]
            i = i + 1
        towers_pos = last_house_covered_by_left_side
        Towers.add(tower_pos)
        reach = tower_pos + R
        while i <= n and P[i] <= reach:
            i = i + 1
    return Towers
```

Let $G = \{g_1, g_2, \dots, g_k\}$ be the set of tower positions chosen by greedy

Let $O = \{o_1, o_2, \dots, o_m\}$ be the optimal set of positions

↳ The greedy algorithm places the first tower g_1 at the furthest possible point that still covers P[1]. Any optimal solution O must also cover P[1] using some tower o_1. Because g_1 is the furthest possible position to the right that covers P[1], we know $o_1 \leq g_1$

↳ If we replace o_1 with g_1 in O, the new set O' still covers p[1] and potentially covers more houses to the right than o_1 did, because $g_1 \geq o_1$

↳ Since we can replace each optimal solution with a greedy choice without increasing the number of towers or losing coverage, the greedy solution is optimal ($k = m$)

Sorting: $O(n \log n)$ to sort house positions Total complexity: $O(n \log n)$

Greedy Pass: While loops traverse the array P exactly once, which is $O(n)$

2.

Recursive Formula: $C[i] = \max(C[i-1], 1 + C[p[i]])$

↳ $C[i-1]$: optimal solution EXCLUDING activity i

↳ $1 + C[p[i]]$: optimal solution INCLUDING activity i

↳ $p[i]$: Largest index $j < i$ such that $f_j \leq s_i$ (latest activity compatible with i)

```
DP-Activity-Selection(S, f, n):
    p = array of size n (all 0)
    for i=1 to n:
        for j = i-1 down to 1:
            if f[j] <= S[i]:
                p[i] = j
                break
    C = array of size n+1
    C[0] = 0
    for i = 1 to n:
        C[i] = max(C[i-1], 1 + C[p[i]])
    return C[n]
```

Precomputing: $O(n^2)$ using the nested loops above. If Binary Search is used (finish times are sorted), this is $O(n \log n)$

Filling Table: A single loop runs n times with $O(1)$ operations per step

Total complexity: $O(n^2)$ or $O(n \log n)$ depending on the $p[i]$ implementation.